

```

<!doctype html>
<html lang="ja">
<head>
<meta charset="utf-8" />
<meta name="viewport" content="width=device-width,initial-scale=1" />
<title>MBTIで行く、京都おすすめスポット</title>
<style>
  :root{font-family:system-ui,-apple-system,"Segoe UI",Roboto,"Hiragino Kaku Gothic ProN",Meiryo,Arial; color:#222;}
  body{margin:0;padding:24px;background:#fbfaf8}
  header{display:flex;gap:12px;align-items:center; margin-bottom:18px;}
  h1{font-size:20px;margin:0}
  .card{background:white;border-radius:12px;padding:16px;box-shadow:0 6px 18px
rgba(15,15,15,0.06);max-width:900px}
  label{display:block;font-size:13px;margin-bottom:6px;color:#444}
  select,input[type="text"]{padding:10px;border-radius:8px;border:1px solid
#e0dcd8;width:100%;box-sizing:border-box}
  button{padding:10px 14px;border-
radius:8px;border:0;background:#2f7cff;color:white;font-
weight:600;cursor:pointer;margin-top:10px}
  button:active{transform:translateY(1px)}
  .result{margin-top:16px}
  .spot{padding:12px;border-radius:10px;border:1px solid #f0ece9;margin-
bottom:10px;background:linear-gradient(180deg,#fff,#fbfdff)}
  .spot h3{margin:0 0 6px 0}
  .muted{color:#666;font-size:13px}
  .controls{display:flex;gap:8px;margin-top:10px;flex-wrap:wrap}
  .small{font-size:13px;padding:8px 10px}
  footer{margin-top:18px;color:#666;font-size:13px}
  @media(min-width:760px){body{padding:40px} .card{padding:22px}}
</style>
</head>
<body>
  <main class="card" role="main">
    <header>
      <h1>MBTIで行く、京都おすすめスポット</h1>
    </header>

    <div>
      <label for="mbti">MBTI を入力（例：INTJ, ENFP など） </label>
      <input id="mbti" placeholder="例: INFP" maxlength="4" />
      <div class="controls">
        <button id="goBtn">おすすめを見る</button>

```

```
<button id="randomBtn" class="small">ランダムMBTIで試す</button>
<select id="prefSelect" class="small" title="好みを選ぶ">
  <option value="any">おすすめの観点: 自動</option>
  <option value="history">歴史・寺社</option>
  <option value="nature">自然・景観</option>
  <option value="food">グルメ / 市場</option>
  <option value="night">夜の町歩き</option>
  <option value="quiet">静かに散策</option>
</select>
</div>
```

```
<div id="result" class="result" aria-live="polite"></div>
```

```
<footer>
```

※このアプリは簡易デモです。スポット情報は概略の紹介です。写真・詳細は後で追加できます。

```
</footer>
```

```
</div>
```

```
</main>
```

```
<script>
```

```
/* ===== データ部分: MBTI -> カテゴリ -> スポット一覧 (例) =====
```

```
※ここを編集して好きなスポットや説明を追加してください */
```

```
const archetypeMap = {
  // 例: INTJ, INTP = 思索型 -> 静かな名所、庭園、寺院
  strategist: {types:['INTJ','INTP'], label:'思索・静寂を好む'},
  explorer:   {types:['ENFP','ENTP'], label:'自由で発見を楽しむ'},
  artist:     {types:['ISFP','INFP','INFJ'], label:'美・情緒・散策'},
  social:     {types:['ESFP','ENFJ','ESFJ','ENTJ'], label:'賑わい・グルメ・夜の町'},
  planner:    {types:['ISTJ','ISFJ','ESTJ','ISTP'], label:'計画的・名所巡り'}
};
```

```
// 京都スポットサンプル (短い説明つき)
```

```
const spots = {
  history: [
    {name:'清水寺', desc:'京都を代表する寺院。清水の舞台からの眺めが有名。'},
    {name:'二条城', desc:'江戸時代の将軍の政治拠点。障壁画や庭園も見どころ。'},
    {name:'金閣寺（鹿苑寺）', desc:'金箔で覆われた楼閣が池に映える名所。'}
  ],
  nature: [
    {name:'嵐山（竹林の小径）', desc:'渡月橋周辺の自然と竹林。ゆったり散策に最適。'},
    {name:'哲学の道', desc:'桜並木や小さな寺社が続く散歩道。季節の移ろいが美しい。'},
    {name:'鴨川沿い', desc:'川沿いの散歩や休憩にぴったり。地元の雰囲気味わえる。'}
  ]
};
```

```

],
food: [
  {name:'錦市場', desc:'「京都の台所」。京料理の素材やお土産が揃う。食べ歩きOK。'},
  {name:'祇園周辺・花見小路', desc:'伝統的な町並みと和食・甘味処が豊富。夜は雰囲気がある。'},
],
  {name:'京料理の小さな町家店', desc:'静かな町家で味わう伝統料理（要予約の店もある）。'}
],
night: [
  {name:'祇園・先斗町（ぼんとちょう）', desc:'夜の料亭や小さな飲食店が並ぶ繁華街。'},
  {name:'鴨川ナイトウォーク', desc:'ライトアップと川風が心地よい夜の散歩コース。'},
  {name:'夜の清水寺（特別拝観時）', desc:'期間限定でライトアップされることがある。'}
],
quiet: [
  {name:'銀閣寺（慈照寺）', desc:'枯山水と落ち着いた庭園。静かに座って景色を楽しめる。'},
  {name:'東福寺の庭園', desc:'紅葉の名所としても有名。平日なら落ち着いて見られる。'},
  {name:'桂離宮（要事前予約）', desc:'日本庭園の傑作。一般公開は限られるが必見。'}
]
];

```

```

// MBTI を簡単にアーキタイプへマッピングする関数
function mapMbtiToArchetype(mbti){
  if(!mbti || mbti.length < 4) return null;
  mbti = mbti.toUpperCase().trim();
  // まず完全一致で探す
  for(const [key,val] of Object.entries(archetypeMap)){
    if(val.types.includes(mbti)) return key;
  }
  // 部分文字に基づく簡易判定（I/E, N/S, F/T）
  const first = mbti[0];
  const second = mbti[1];
  const third = mbti[2];
  const fourth = mbti[3];
  if(first === 'I'){
    if(third === 'N' || third === 'F') return 'artist';
    return 'planner';
  } else {
    // E
    if(third === 'N') return 'explorer';
    if(fourth === 'J') return 'social';
    return 'explorer';
  }
}

```

```

// MBTI アーキタイプに基づいてスポットカテゴリを選ぶ
const archetypeToCategories = {
  strategist: ['quiet','history','nature'],
  explorer:  ['nature','food','night'],
  artist:    ['history','nature','quiet'],
  social:    ['food','night','history'],
  planner:   ['history','quiet','food']
};

function buildRecommendations(archetype, prefOverride){
  const cats = archetypeToCategories[archetype] || ['history','nature','food'];
  // ユーザーの好み設定されていれば先頭に置く
  if(prefOverride && prefOverride !== 'any'){
    const mapPref = {history:'history', nature:'nature', food:'food', night:'night',
quiet:'quiet'};
    const p = mapPref[prefOverride];
    if(p){
      // 移動して先頭に
      const filtered = [p, ...cats.filter(c=>c!==p)];
      return pickFromCategories(filtered);
    }
  }
  return pickFromCategories(cats);
}

function pickFromCategories(catList){
  const results = [];
  for(const cat of catList){
    const pool = spots[cat] || [];
    // pick first two unique spots
    for(const s of pool.slice(0,2)){
      if(!results.some(r=>r.name===s.name)) results.push({...s,category:cat});
      if(results.length >= 6) break;
    }
    if(results.length >= 6) break;
  }
  return results;
}

/* ===== UI ハンドリング ===== */
const mbtiInput = document.getElementById('mbti');
const goBtn = document.getElementById('goBtn');
const randomBtn = document.getElementById('randomBtn');

```

```

const resultDiv = document.getElementById('result');
const prefSelect = document.getElementById('prefSelect');

function showRecommendations(mbti, pref){
  const archetype = mapMbtiToArchetype(mbti);
  const archeLabel = archetype ? (archetypeMap[archetype] ?
archetypeMap[archetype].label : archetype) : '不明';
  const recs = archetype ? buildRecommendations(archetype, pref) : [];

  resultDiv.innerHTML = `
    <div class="muted">入力: <strong>${mbti.toUpperCase()}</strong> → 分類: <strong>${
{archeLabel}</strong></div>
    <div style="margin-top:10px;">
      ${recs.length ? recs.map(r => `
        <div class="spot">
          <h3>${r.name}</h3>
          <div class="muted">${r.category}</div>
          <p style="margin:8px 0 0">${r.desc}</p>
        </div>
      `).join("") : `<p class="muted">入力が不明です。例: ENFP, ISTJ のように4文字で入力して
みてください。</p>`}
    </div>
  `;
}

goBtn.addEventListener('click', ()=>{
  const raw = mbtiInput.value || "";
  showRecommendations(raw, prefSelect.value);
});

// ランダムMBTI
randomBtn.addEventListener('click', ()=>{
  const examples = ['INTJ','ENFP','ISFP','ENTJ','INFP','ESFP','ISTJ','ENFJ','ISTP','INFJ'];
  const pick = examples[Math.floor(Math.random()*examples.length)];
  mbtiInput.value = pick;
  showRecommendations(pick, prefSelect.value);
});

// Enterで実行
mbtiInput.addEventListener('keydown', (e)=>{ if(e.key === 'Enter') goBtn.click(); });

// 初回サンプル表示
showRecommendations('INFP', 'any');

```

</script>  
</body>  
</html>